Factor-Graph Model Predictive Control and Terminal-Measurement Factor MPC

IPN_MPC

August 18, 2026

Abstract

This note introduces model predictive control (MPC) expressed as nonlinear factor-graph optimization and a reduced-variable variant based on the `TerminalAccelerationGyroMeasurementFactor`. The conventional formulation represents future states and controls as optimization variables. The terminal-measurement formulation instead rolls the current state forward through prefixes of the control sequence and compares each prediction directly with a fixed trajectory set point. The latter removes future-state variables, uses analytical chain-rule Jacobians, employs a discrete Riccati terminal weight for local stability, and enforces hard control bounds with a constrained quadratic-programming refinement.

1 MPC as factor-graph optimization

For a horizon of length N, let x_k denote the state, u_k the control input, and x_k^{ref} the reference state. A standard discrete-time MPC problem is

$$\min_{x_{1:N},u_{0:N-1}} \quad \sum_{k=1}^{N-1} \frac{1}{2}\|x_k - x_k^{\text{ref}}\|_{Q_k}^2 + \sum_{k=0}^{N-1} \frac{1}{2}\|u_k - u_k^{\text{ref}}\|_{R_k}^2 + \frac{1}{2}\|x_N - x_N^{\text{ref}}\|_P^2, \tag{1}$$

$$\text{subject to} \quad x_{k+1} = f(x_k, u_k), \qquad k = 0, \dots, N-1, \qquad u_{\min} \le u_k \le u_{\max}. \tag{2}$$

Here $\|e\|_W^2 = e^\mathsf{T} W e$. In a factor graph, every quadratic term is a factor and every unknown state or control is a variable node. A Gaussian factor with covariance Σ contributes

$$\frac{1}{2} e^\mathsf{T} \Sigma^{-1} e, \tag{3}$$

so the desired weight matrix is supplied to GTSAM as the information matrix $W = \Sigma^{-1}$.

Typical graph components are:

- a strong prior on the measured current state x_0;
- dynamics factors joining adjacent states x_k, u_k, x_{k+1};
- tracking factors on x_k;
- control regularization and smoothness factors on u_k and (u_{k-1}, u_k);
- a terminal tracking factor on x_N; and
- optional obstacle, safety, and actuator constraints.

After optimization, MPC applies only u_0, advances the simulator or vehicle, shifts the previous solution to form a warm start, and solves the horizon again.

2 Acceleration–angular-speed model

The terminal-factor applications use

$$x_k = (R_k, p_k, v_k), \qquad R_k \in \mathrm{SO}(3), quadp_k, v_k \in \mathbb{R}^3, \tag{4}$$

and the six-dimensional control

$$u_k = \begin{bmatrix} a_k \\ \omega_k \end{bmatrix} \in \mathbb{R}^6, \tag{5}$$

where a_k is world-frame acceleration and ω_k is body angular speed. With sampling time Δt, the rollout is

$$p_{k+1} = p_k + v_k \Delta t + \frac{1}{2} a_k \Delta t^2, \; v_{k+1} \qquad = v_k + a_k \Delta t, \; R_{k+1} = R_k \,\mathrm{Exp}(\omega_k \Delta t). \tag{6}$$

The optimized kinematic command is converted to collective thrust and torque before it is applied to the full quadrotor simulator.

3 State-variable terminal rollout factor

The `TerminalAccelerationGyroFactor` connects the initial state, a control prefix, and an optimized future state:

$$\{X_0, V_0, U_0, \ldots, U_{j-1}, X_j, V_j\}. \tag{7}$$

Starting from x_0, define

$$\widehat{x}_j = f^{(j)}(x_0, u_0, \ldots, u_{j-1}). \tag{8}$$

Its residual is

$$e_j = \begin{bmatrix} p_j - \widehat{p}_j \\ \mathrm{Log}(\widehat{R}_j^{-1} R_j) \\ v_j - \widehat{v}_j \end{bmatrix} \in \mathbb{R}^9, \qquad E_j = \frac{1}{2} e_j^{\mathsf{T}} Q_j e_j. \tag{9}$$

The factor is a soft multiple-shooting constraint. Separate priors or tracking factors attract X_j, V_j to the desired trajectory. This formulation retains explicit predicted-state variables, which are useful when other factors must attach directly to future states, for example obstacle, sensor, or state-boundary factors.

4 Terminal-measurement factor MPC

4.1 Fixed set point instead of a future-state variable

The `TerminalAccelerationGyroMeasurementFactor` stores a desired pose and velocity as fixed factor measurements. Its variable set is only

$$\{X_0, V_0, U_0, \ldots, U_{j-1}\}; \tag{10}$$

there is no X_j or V_j variable. For the measured set point $z_j = (R_j^{\mathrm{ref}}, p_j^{\mathrm{ref}}, v_j^{\mathrm{ref}})$, the factor residual is

$$e_j = \begin{bmatrix} p_j^{\mathrm{ref}} - \widehat{p}_j \\ \mathrm{Log}(\widehat{R}_j^{-1} R_j^{\mathrm{ref}}) \\ v_j^{\mathrm{ref}} - \widehat{v}_j \end{bmatrix}, \qquad E_j = \frac{1}{2} e_j^{\mathsf{T}} Q_j e_j. \tag{11}$$

The MPC graph adds N prefix factors:

$$\begin{aligned} e_1 &= z_1 - f(x_0, u_0), \\ e_2 &= z_2 - f^{(2)}(x_0, u_0, u_1), \\ &\vdots \\ e_N &= z_N - f^{(N)}(x_0, u_0, \ldots, u_{N-1}). \end{aligned} \tag{12}$$

Consequently, the unknowns are the current state and control sequence, rather than an entire state–control trajectory.

4.2 Comparison of the two formulations

Property	State-variable factor	Measurement factor
Future X_j, V_j variables	Required	Removed
Set point	Separate prior/factor	Stored in factor
Attach factors to future state	Direct	Requires rollout-dependent factor
Graph size	Larger	Smaller
Tracking residual	Future state minus rollout	Set point minus rollout
Application	`terminal_acceleration_gyro_mpc`	`terminal_acceleration_gyro_setpoint`

Table 1: Comparison of terminal acceleration–gyro MPC formulations.

5 Analytical chain-rule Jacobians

Numerically differentiating every prefix repeats many rollouts and is unnecessarily expensive. The measurement factor therefore uses analytical derivatives. For a j-step rollout,

$$\widehat{v}_j = v_0 + \Delta t \sum_{k=0}^{j-1} a_k, \tag{13}$$

$$\widehat{p}_j = p_0 + j\Delta t\, v_0 + \Delta t^2 \sum_{k=0}^{j-1} \left(j - k - \frac{1}{2} \right) a_k. \tag{14}$$

Thus,

$$\frac{\partial e_p}{\partial v_0} = -j\Delta t I, \qquad \frac{\partial e_v}{\partial v_0} = -I, \tag{15}$$

$$\frac{\partial e_p}{\partial a_k} = -\left(j - k - \frac{1}{2} \right) \Delta t^2 I, \qquad \frac{\partial e_v}{\partial a_k} = -\Delta t I. \tag{16}$$

For rotation, write

$$\widehat{R}_j = R_0 E_0 E_1 \cdots E_{j-1}, \qquad E_k = \mathrm{Exp}(\omega_k \Delta t). \tag{17}$$

Let $S_k = E_{k+1} \cdots E_{j-1}$ be the rotation suffix after E_k. A local perturbation of E_k is transported to the final rotation through $\mathrm{Ad}_{S_k^{-1}}$. Combining this transport with the exponential-map Jacobian and the Jacobian of $\mathrm{Log}(\widehat{R}_j^{-1} R_j^{\mathrm{ref}})$ gives

$$\frac{\partial e_R}{\partial \omega_k} = J_{\mathrm{Log}}\, J_{\mathrm{between}}\, \mathrm{Ad}_{S_k^{-1}}\, J_{\mathrm{Exp}}(\omega_k \Delta t) \Delta t. \tag{18}$$

All suffixes are accumulated in one backward pass. This avoids finite differences and keeps the default 20-step MPC comfortably below its 10-ms solve-time target on the development machine.

6 Terminal weight for local stability

Near zero tracking error, use the local vector

$$\xi_k = \begin{bmatrix} \delta p_k & \delta\theta_k & \delta v_k \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^9, \qquad \nu_k = \begin{bmatrix} \delta a_k & \delta\omega_k \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^6. \tag{19}$$

The linearized model is

$$\xi_{k+1} = A\xi_k + B\nu_k, \tag{20}$$

with

$$A = \begin{bmatrix} I & 0 & \Delta t I \\ 0 & I & 0 \\ 0 & 0 & I \end{bmatrix}, \qquad B = \begin{bmatrix} \frac{1}{2}\Delta t^2 I & 0 \\ 0 & \Delta t I \\ \Delta t I & 0 \end{bmatrix}. \tag{21}$$

Given positive-definite stage weights Q and R, the terminal information matrix P is the stabilizing solution of the discrete algebraic Riccati equation

$$P = Q + A^{\mathsf{T}}PA - A^{\mathsf{T}}PB(R + B^{\mathsf{T}}PB)^{-1}B^{\mathsf{T}}PA. \tag{22}$$

The feedback gain is

$$K = (R + B^{\mathsf{T}}PB)^{-1}B^{\mathsf{T}}PA. \tag{23}$$

The implementation verifies that $\rho(A - BK) < 1$, that the normalized Riccati residual is small, and that the Lyapunov terminal cost strictly decreases under the local feedback. This establishes local stability of the unconstrained linearized error dynamics. A full constrained nonlinear MPC proof additionally requires a feasible invariant terminal set.

7 Hard control-input bounds

Standard GTSAM nonlinear optimizers do not directly impose inequality constraints. The set-point application first computes a nonlinear Levenberg–Marquardt solution and then applies a sequential-quadratic-programming refinement using `gtsam_unstable::QPSolver`. At an iterate u_k, the QP increment δu_k obeys

$$\delta u_{k,i} \le u_{\max,i} - u_{k,i}, \tag{24}$$

$$-\delta u_{k,i} \le u_{k,i} - u_{\min,i}. \tag{25}$$

These are exact box constraints for the linear control variables. The QP is initialized with the projected feasible increment

$$\delta u_k^{(0)} = \mathrm{clip}(u_k, u_{\min}, u_{\max}) - u_k. \tag{26}$$

Before u_0 is applied, the complete optimized horizon is checked against the configured bounds with a tolerance of 10^{-8}.

8 Receding-horizon algorithm

At each control cycle, the set-point application performs the following steps:

1. Read the current simulated pose and velocity and add strong priors on X_0, V_0.
2. Generate N circle-trajectory set points.
3. Add one terminal-measurement factor for every control prefix.
4. Add control regularization and control-smoothness factors.

5. Use the DARE matrix P for the final factor and Q for intermediate factors.
6. Solve the nonlinear least-squares problem from the shifted warm start.
7. Refine the solution with hard box-constrained QPs.
8. Verify feasibility and apply only u_0.
9. Shift the control sequence and repeat at the next measurement.

9 Configuration and execution

The principal settings are stored in `config/terminal_acceleration_gyro_mpc.yaml`. The control ordering is

$$U(k) = [a_x, a_y, a_z, \omega_x, \omega_y, \omega_z]^\mathsf{T}. \tag{27}$$

For example,

```
horizon: 20
dt: 0.10
terminal_position_sigma: 0.02
terminal_rotation_sigma: 0.02
terminal_velocity_sigma: 0.05
control_min: [-4.0, -4.0, -4.0, -2.0, -2.0, -2.0]
control_max: [ 4.0,  4.0,  4.0,  2.0,  2.0,  2.0]
hard_control_constraint_iterations: 2
```

Build and run the two formulations with

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel

./build/terminal_acceleration_gyro_mpc \
  config/terminal_acceleration_gyro_mpc.yaml

./build/terminal_acceleration_gyro_setpoint_mpc \
  config/terminal_acceleration_gyro_mpc.yaml
```

Append `--headless` to either command for automated testing without Pangolin visualization.

10 Implementation map

Purpose	Source
Terminal factor declarations	`include/ipn_mpc/control/dynamics_control_factor.h`
Factor residuals and Jacobians	`src/control/dynamics_control_factor.cpp`
DARE terminal weight	`src/control/lqr_terminal_weight.cpp`
State-variable MPC	`apps/terminal_acceleration_gyro_mpc.cpp`
Set-point MPC	`apps/terminal_acceleration_gyro_setpoint_mpc.cpp`
Factor tests	`tests/terminal_dynamics_factor_test.cpp`
Terminal stability test	`tests/lqr_terminal_weight_test.cpp`